People`s Democratic Republic of Algeria
The Ministry of Higher Education and Scientific Research

University Of Science And Technology Houari Boumediene
Faculty of Computer Science

# Software Testing and Quality Assurance

## Software Quality

Fourth year in Computer Science engineering

By: PhD. HAOUARI Ahmed Taha
Info.ahmed.taha@gmail.com

2025/2026

# 1

# What do we mean by Quality

It work and look Good !!

# Quality

Let see what IA say :

**ChatGpt:**

In everyday life, **quality** simply means how good something is. When people talk about quality, they usually mean that a product or service works well, lasts long, looks or feels well-made, and satisfies their expectations. A high-quality item is one that performs reliably, does not easily break or disappoint, and gives a sense of value for the money paid. In this common sense, quality is mostly based on personal experience and perception rather than on formal standards or measurements.

**Gemini**

In simple terms, **quality** is the bridge between a promise and a reality. Every product or service makes a "promise" to the person who uses it; if the reality of using that product matches or exceeds what was promised,

# Quality

**Your professor View:**
Quality may be defined as a subjective evaluation of how effectively a product, service, or environment meets expectations within a given context. It varies according to individual needs and situational factors. For instance, in the context of travel, comfort and affordability are often key determinants of transportation quality. Thus, quality can be perceived as the level of satisfaction achieved in relation to the associated cost.

**What’s your view ???**
….

# Quality

"Quality is the degree to which a set of built-in characteristics of an object fulfills requirements."
— **ISO 9000:2015**, *Quality management systems — Fundamentals and vocabulary*.

"Quality is fitness for use.“
— **Juran, J. M. (1951)**, *Quality Control Handbook*.

"Quality is conformance to requirements.“
— **Crosby, P. B. (1979)**, *Quality is Free*.

"Quality lies in the eyes of the beholder.“
— **Garvin, D. A. (1984)**, "What Does 'Product Quality' Really Mean?" *Sloan Management Review*.
"Quality is a characteristic or attribute of something. As an attribute of an item, quality refers to measurable characteristics—things we can compare to known standards such as size, color, properties, and tolerances.“
— **Pressman, R. S. (2005)**, *Software Engineering: A Practitioner's Approach*.

# 2

# Software

Program ??

# Software Characteristics

**Unique Product and replicable**

Software is typically **developed once for a specific purpose**. While it can be replicated easily, each software system is generally **designed and built to meet particular requirements**, making it unique in terms of functionality, design, and implementation.

Example: A custom enterprise resource planning system is built for one organization and tailored to its needs.

**Non-Wearing / Inexhaustible**

Unlike hardware, software **does not wear out physically**. Failures are not caused by physical degradation but **originate from defects, poor design, or changing requirements**.

Example: A program may crash because of a bug, not because it “used up” like a machine part.

**Complex**

Software is inherently **complex** because it is created to **solve complex problems** that humans cannot easily manage manually. Its complexity arises from algorithms, logic, interdependencies, and the variety of possible user interactions.

Example: An operating system must manage thousands of processes, memory allocations, and hardware devices simultaneously.

**Intangible / Invisible**

Software is **intangible**, meaning it has **no physical form**. Its development is a purely intellectual activity, making it difficult to observe, touch, or measure directly. This intangibility makes **perceiving and evaluating software quality more challenging** than evaluating physical products.

Example: Code quality or software maintainability cannot be directly “seen” like a physical product.

**Technically Immature / Still Evolving**

Despite significant advances in software engineering, software development often remains **partially artisanal**, with creativity and human expertise playing a central role. Many processes, tools, and techniques are still evolving, and **standardization and maturity vary across domains**.

Example: Large-scale AI systems still rely on expert judgment for architecture and optimization.

Software is a unique engineering product with characteristics that make its **development, evaluation, and quality assurance distinct** from hardware. Its uniqueness, complexity, intangibility, and technical immaturity explain why **software quality assessment requires specialized models, metrics, and practices**. Understanding these characteristics is a prerequisite for defining and measuring software quality effectively.

# Software Development Plan

A **Software Development Plan (SDP)** provides a structured framework for managing the creation of a software product. It describes the objectives, processes, resources, and responsibilities required to successfully complete the project. A complete development plan typically includes the following components:

**Description of the Software Product at Multiple Levels**

The SDP should provide a **detailed description of the software to be developed**, including:

Individual programs or modules

Associated documentation (requirements, design, test plans, user manuals)

This multi-level description ensures that **all components and deliverables are clearly defined** before development begins.

**Resources (Hardware, Software, and Methods)**

The plan must identify all **resources required for development**, including:

- Hardware: servers, workstations, network devices
- Software: development tools, compilers, frameworks
- Methods and techniques: coding standards, design methodologies, testing strategies
- Tools: project management software, version control systems

This ensures that the development team has **everything necessary to execute the project efficiently**.

**Progress Monitoring Tools**

The plan should define **tools and methods for tracking project progress**, including:

Detailed planning schedules

Milestones and deadlines by using Gantt charts, calendars, or other tracking mechanisms

This allows the team and management to **monitor development status and take corrective action when necessary**.

**Life Cycle Breakdown and Task Assignment**

The SDP should **divide the software life cycle into distinct phases**, such as:

Requirements analysis

System design

Implementation / coding

Testing and validation

Deployment and maintenance

For each phase, the plan should:

Define specific tasks to be completed

Identify responsible personnel for each task

This ensures **clarity in responsibilities and accountability**.

**Project Management Methods**

The SDP should specify the **management methods used to control the project**, including:

Risk management techniques

Change management processes

Communication plans

Resource allocation and optimization

This ensures **project objectives are met on time, within budget, and according to quality standards**.

**Key Milestones and Client Involvement**

The plan should identify **critical points in the project** where:

Deliverables are reviewed or validated

Client intervention or feedback is required

Decisions must be made to continue, revise, or adjust the project

This ensures **alignment between the development team and stakeholders** throughout the project.

# Observations on Software Development

Software development presents unique challenges and costs compared to hardware production. Several key observations can be made regarding the development and lifecycle of software:

The **cost of developing software often surpasses the cost of the hardware** on which it runs. While hardware requires physical materials, software costs are dominated by human effort, design, coding, and testing.

Typical cost distribution in software projects is approximately:
**20%** for coding and individual debugging
**30%** for system and module design
**50%** for integration testing and validation
This illustrates that **testing and design constitute the majority of development effort**, not just coding.

# Observations on Software Development

Software generally has a **long operational life**, often ranging from **7 to 15 years**. During this period, maintenance and evolution contribute significantly to overall costs.

The **cost of corrective and evolutionary maintenance** typically constitutes the largest portion of the total software cost. Maintenance includes
Correcting defects discovered post-deployment
Adapting the system to new requirements or environments
Enhancing performance or usability
This highlights the **importance of building the best possible software from the start**.

**Errors Introduced Early Are Most Expensive,** More than **half of the errors detected during testing** originate from mistakes made in the early phases of development (requirements and design).The **later an error is discovered**, the more

# 3

# Software Quality

He has to do what he suppose to do

# Definition

*the degree to which a software product satisfies stated and implied needs when used under specified conditions*. This definition emphasizes that software quality depends not only on explicitly defined requirements but also on implicit expectations of users and the environment in which the software operates.
**ISO/IEC 25010 (2011). *Systems and Software Engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.***

**Ian Sommerville** defines software quality as the extent to which a software system meets its specified functional and non-functional requirements and satisfies user needs in an effective and reliable manner.
**Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.**

# Definition

According to **Roger S. Pressman**, software quality is *conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software*.

**Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.**

# Definition

According to the concepts discussed previously:
Software quality is **the application of systematic methods, tools, and processes** **throughout the software development lifecycle** to produce a software system that **meets the client's requirements** and **follows a planned development process.**
Software quality is **not only about the absence of defects,** it also ensures that the software performs **its intended functions correctly**, operates reliably, uses resources efficiently, and can be maintained and adapted over time.
From this perspective, software quality represents an engineering challenge, where achieving a reliable and effective product **requires careful planning, adherence to standards, and continuous evaluation at every stage of development.**

# Historical Disasters Caused By poor quality

**Downing of an Airbus by the USS Vincennes** In **1988**, the U.S. Navy cruiser USS Vincennes (CG-49) mistakenly shot down the civilian aircraft Iran Air Flight 655.
The radar detection system displayed **confusing and cryptic information**.
The operators misinterpreted the aircraft as a hostile military plane.
As results, 290 passengers and crew were killed.
**The quality issue are**
Poor **human–computer interface design,**
Misinterpretation of system outputs.

**Patriot Missile Failure** During the Gulf War, the missile defense system MIM-104 Patriot failed to intercept an incoming Iraqi missile. **Caused by** An **arithmetic rounding error in time calculation**. The system accumulated timing errors after running continuously for many hours.
**As Result** The missile location was miscalculated. And 28 U.S. soldiers were killed.
**The quality issue is** Numerical precision and **floating-point computation errors**.(not understanding the the hardware

# Historical Disasters Caused By poor Quality

**London Ambulance Service Computer Aided Dispatch System:** The emergency dispatch system of the London Ambulance Service attempted to automate ambulance allocation.
**Caused by** The software system was **poorly tested**. In reality, It could not handle the volume of emergency calls.
**As Result** Ambulances were not dispatched correctly. Several patients died due to delayed assistance.
**Everything was due to** Poor **software testing and project management**.

**False Missile Attack Alert**
On June 3, 1980, the North American Aerospace Defense Command reported that the United States was under missile attack. Everything is due to A **faulty computer chip** sent incorrect signals indicating hundreds of incoming missiles. Thus, The U.S. nuclear forces were briefly **placed on high alert.**
It cause by Lack of proper **input validation and redundancy checks**.

# Historical Disasters Caused By poor Quality

Ariane 5 was developed as the successor to Ariane 4 for launching satellites into Earth orbit.
During its first mission, Ariane 5 Flight 501 exploded **37 seconds after launch** because incorrect information was sent to the flight control system.
The failure occurred when the software attempted to convert a **64-bit floating-point number into a 16-bit signed integer**, causing a numeric overflow.

Since no **exception handling** was implemented, the inertial reference system stopped functioning and transmitted invalid data. The faulty code had been **reused from Ariane 4**, where the lower acceleration never produced such large values, so the error had never appeared in tests or real flights.

They didn’t update the code to the newer spec, and poor testing lead to 370 million dollar of lost in 1996 which equivalent to almost a billion in 2026

# Factors Causing Poor Software Quality

**Poor Specification:**
It occur when the **software requirements are vague**, **incomplete, ambiguous, or unstable.** If the system requirements are not clearly defined, developers may interpret them differently, which leads to incorrect implementations.
Frequent changes in requirements can also create inconsistencies and make the system difficult to design and maintain. As a result, the final **product may not meet the user's real needs.**

**Poor Estimation:**
Incorrect estimation happens when the **time, cost, or resources required for the project are badly evaluated**. Important tasks may be forgotten, or the complexity of the system may be underestimated. This can lead to unrealistic deadlines, insufficient resources, and excessive pressure on the development team, which often results in lower software quality.

# Factors Causing Poor Software Quality

**Poor Task Distribution**

Software projects require a **well-organized distribution of responsibilities**. If tasks are not assigned according to the skills of team members, or if project constraints are ignored, the development process becomes inefficient.

Additionally, if deviations from the plan are not detected early, problems can accumulate and become **more difficult and expensive to fix later.**

**Poor Technical Implementation**

Technical issues may arise during **coding, testing, and documentation**. Poor programming practices, insufficient testing, or incomplete documentation can introduce defects into the software.

Without proper verification and validation processes, **these errors may remain undetected until the system is already deployed.**

# Factors Causing Poor Software Quality

**Human Factors**
Human issues can significantly affect software quality.
Examples include
**poor distribution of work,**
**conflicts between team members,**
**lack of communication, or information being withheld**.
Another common problem is the **lack of experience of the project manager**, which can lead to poor decision-making, inadequate planning, and ineffective coordination of the development team.

# Main Software Quality Models

**McCall's Quality Model (1977)**

- Correctness
- Reliability
- Efficiency
- Integrity
- Usability
- Maintainability
- Flexibility
- Testability
- Portability
- Reusability
- Interoperability

**Boehm's Quality Model (1978)**

- Usability
- Efficiency
- Reliability
- Portability
- Testability
- Understandability
- Flexibility
- Modifiability
- Human Engineering
- Conciseness
- Reusability
- Integrity

**FURPS Model (IBM, 1992)**

- **F** unctionality
- **U** sability
- **R** eliability
- **P** erformance
- **S** upportability

**ISO/IEC 9126 (1991)**

- Functionality
- Reliability
- Efficiency
- Maintainability
- Portability

**ISO/IEC 25010 (2011)**

- Functional Suitability
- Performance Efficiency
- Compatibility
- Usability NEW
- Reliability NEW
- Security NEW
- Maintainability
- Portability

First Characteristics | Added / Updated Characteristics | ISO/IEC 9126 (1991)

# McCall's Software Quality Model

James McCall proposed one of the **earliest structured software quality models** in 1977. The model focuses on **defining software quality from the user's perspective** and linking it to **measurable metrics**. It organizes software quality into **three main categories**, with **11 factors** in total, and identifies metrics to evaluate them.
The main categogies are:

**1. Product Operation**

This category evaluates how well the software performs its intended functions and interacts with users. Factors include:

**Correctness**: Does the software produce correct and expected results?
**Reliability**: Can the software operate under specified conditions without failure?
**Efficiency**: Does it use computing resources (CPU, memory) effectively?
**Integrity**: Is the software protected from unauthorized access or malicious use?
**Usability**: How easy is it for users to operate and learn the system?

# McCall's Software Quality Model

**2. Product Revision**
This category evaluates how easy it is to **modify the software** for corrections, enhancements, or adaptation. Factors include:
**Maintainability**: Can defects or bugs be corrected efficiently?
**Flexibility**: Can the system accommodate new requirements or changes easily?
**Testability**: How easily can the software be tested to ensure correctness?

**3. Product Transition**
This category evaluates how easily software can **move to new environments** or be reused. Factors include:
**Portability**: Can the software run on different platforms or operating systems?
**Reusability**: Can parts of the software be reused in other projects?
**Interoperability**: Can the software work with other systems or components?

# McCall's Software Quality Model

**2. Product Revision**
This category evaluates how easy it is to **modify the software** for corrections, enhancements, or adaptation. Factors include:
**Maintainability**: Can defects or bugs be corrected efficiently?
**Flexibility**: Can the system accommodate new requirements or changes easily?
**Testability**: How easily can the software be tested to ensure correctness?

**3. Product Transition**
This category evaluates how easily software can **move to new environments** or be reused. Factors include:
**Portability**: Can the software run on different platforms or operating systems?
**Reusability**: Can parts of the software be reused in other projects?
**Interoperability**: Can the software work with other systems or components?

# Boehm’s Software Quality Model

Barry Boehm proposed his model in **1978** as an extension of McCall’s Quality Model. Boehm wanted to provide a **more hierarchical and structured view of software quality**, linking **high-level goals** to measurable attributes. The model emphasizes that software quality is **multi-dimensional** and that higher-level quality goals depend on lower-level technical properties.
Boehm’s model organizes software quality into **three hierarchical layers**, connecting **general objectives** to **measurable properties**:

**1. High-Level Characteristics (Qualities for Users)**
These are **overall goals of software quality** from the user’s perspective. They include:
**Utility**: How useful the software is in fulfilling user needs.
**Maintainability**: How easily the system can be modified or updated.
**Portability**: How easily the software can be transferred to other platforms.

These characteristics describe **what the user experiences or requires**.

# Boehm's Software Quality Model

**2. Intermediate Characteristics (Technical Qualities)**
These are **more specific qualities** that support the high-level characteristics. Examples include:
**Reliability**: How dependable the software is during operation.
**Efficiency**: How well the software uses resources like CPU, memory, or storage.
**Human Engineering**: How well the software fits human cognitive and operational needs.
**Understandability**: How easily the software can be read and understood by developers.
These bridge the gap between **user-focused goals and measurable attributes**.

**3. Primitive Characteristics (Measurable Attributes)**
These are **low-level, directly measurable properties** of the software that affect the intermediate characteristics, such as:
**Modularity**: Degree to which components can be separated and recombined.
**Simplicity**: How simple the design and code structure are.
**Testability**: How easy it is to test the software for defects.
**Conciseness**: Efficiency in using code and resources.
These characteristics can be **quantified or assessed** through metrics or analysis.

# ISO/IEC 9126 model

ISO/IEC 9126 was developed to provide an **internationally recognized framework** for evaluating software quality. It builds on previous models like McCall and Boehm but focuses on **measurable characteristics of the software product** itself. The model helps organizations **assess, compare, and improve software quality** systematically.

ISO/IEC 9126 defines **six main quality characteristics(Attribute)**, each with sub-characteristics and metrics to evaluate them.

**1. Functionality**

Functionality refers to the **ability of the software to provide functions that meet stated and implied needs**. Ensures the software **performs the tasks it was designed for correctly and securely**.

**Sub-characteristics:**

**Suitability**: The functions provided meet user needs and are appropriate for the task.

**Accuracy**: The results of software functions are correct and precise.

**Interoperability**: The software can interact with other systems or components.

**Security**: Protection against unauthorized access, ensuring confidentiality, integrity, and non-repudiation.

**Compliance**: Adherence to standards, laws, or regulations relevant to the software.

# ISO/IEC 9126 model

**2. Reliability**

Reliability measures the **ability of the software to maintain its performance under specified conditions for a specified period**.

Focuses on **stability, error handling, and system uptime**, which are critical for mission-critical applications.

**Maturity**: The software has few failures and is dependable during normal use.

**Fault tolerance**: The software continues to operate correctly even when errors occur.

**Recoverability**: The system can recover quickly from failures, minimizing downtime.

**3. Usability**

Usability evaluates **how easy it is for users to understand, learn, and operate the software**.

**Understandability**: How easily users can understand what the software does and how to use it.

**Learnability**: How easy it is for new users to learn how to operate the system.

**Operability**: How easy it is to operate and control the software during normal use.

**Attractiveness**: The user interface is pleasant, intuitive, and user-friendly.

**Compliance**: The system follows ergonomic or human-computer interaction standards.

# ISO/IEC 9126 model

**4. Efficiency**
Efficiency measures the **performance of software relative to the resources it consumes**.

**Time behavior**: The response times and processing times meet user expectations.
**Resource utilization**: The software uses memory, CPU, disk, and network resources efficiently.

**5. Maintainability**
Maintainability evaluates **how easily the software can be modified** to correct faults, improve performance, or adapt to new environments.

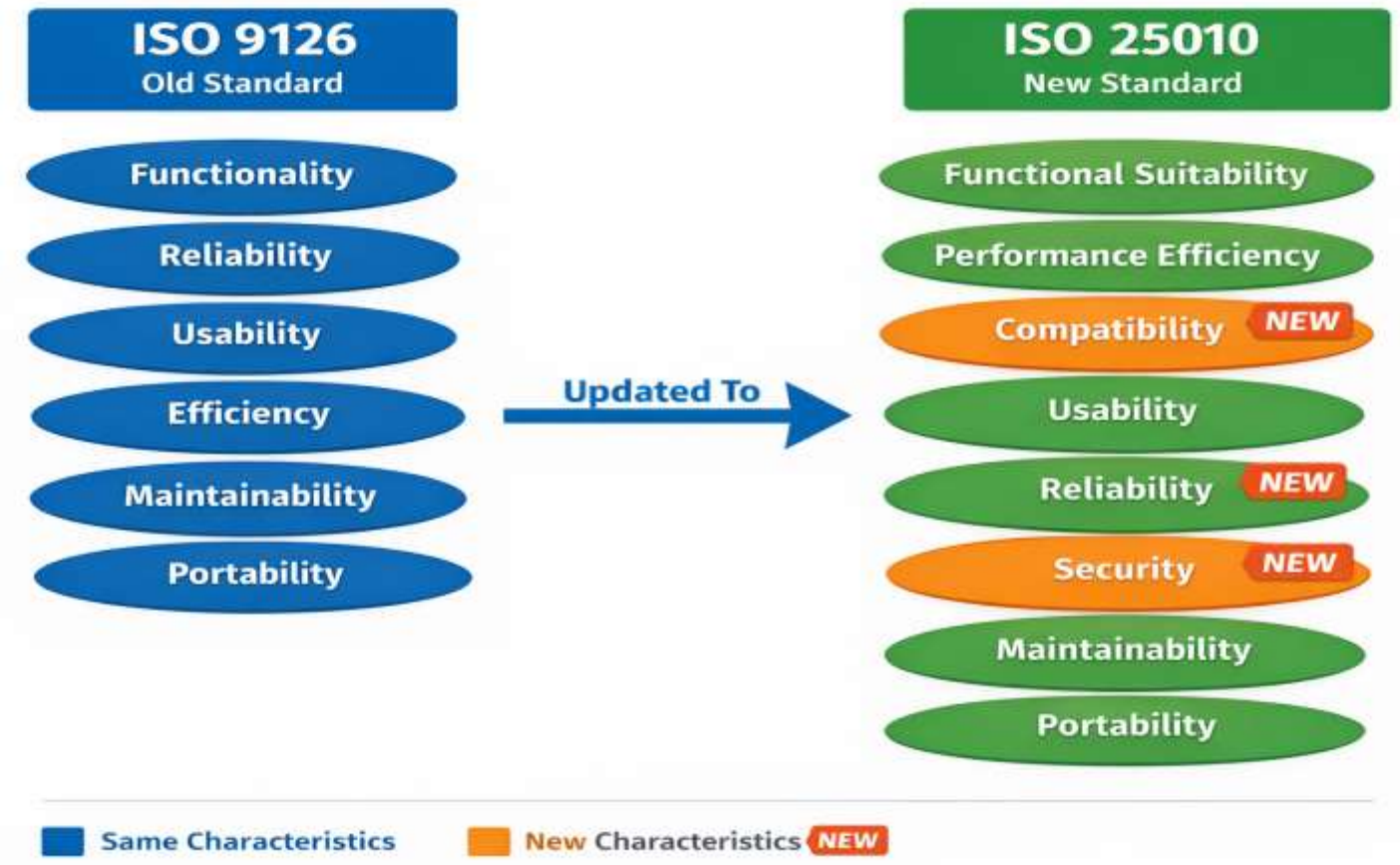
**Analyzability**: How easily the software can be analyzed for defects or causes of failure.
**Changeability**: How easy it is to make modifications without introducing errors.
**Stability**: Changes do not cause unexpected effects in other parts of the system.
**Testability**: How easy it is to test the software after modifications.

# ISO/IEC 9126 model

**6. Portability**
Portability measures the **ability of software to be transferred from one environment to another**.

**Adaptability**: How easily software can adapt to different environments or platforms.
**Installability**: Ease of installation in new environments.
**Conformance**: Compliance with standards or platform requirements.
**Replaceability**: The ability to replace other software in a given environment without issues.

ISO/IEC 9126 provides a **comprehensive hierarchical model**: **6 main characteristics → sub-characteristics → measurable metrics**. It ensures software quality is assessed not only at a high level (like usability or reliability) but also in detail through measurable properties. This makes it a **foundational model for software evaluation and testing**.


ISO 9126
Old Standard
Functionality
Reliability
Usability
Efficiency
Maintainability
Portability
Updated To
ISO 25010
New Standard
Functional Suitability
Performance Efficiency
Compatibility NEW
Usability
Reliability NEW
Security NEW
Maintainability
Portability
Same Characteristics
New Characteristics NEW

# ISO/IEC 25010 model

**2. Security *(new)***

Measures the software's ability to **protect data and resources from unauthorized access** and ensure trustworthiness.

**Confidentiality** – Protection of information from unauthorized access.
**Integrity** – Ensures data and functions are not altered improperly.
**Non-repudiation** – Actions cannot be denied; accountability is maintained.
**Accountability** – All actions can be traced to responsible users or processes.
**Authenticity** – Verifies the identity of users, data, or processes.

**3. Compatibility *(new)***

Evaluates the software's ability to **coexist and interoperate with other systems**.

**Co-existence** – Software can operate in a shared environment without conflicts.
**Interoperability** – Ability to exchange and use information with other software systems.

**Accessibility:** Usability now explicitly includes accessibility, ensuring software is inclusive for users with disabilities.

# Quality Attribute Trade-off

| Priority Goal | Potential Trade-off | Why it happens |
|---|---|---|
| **Usability** | **Maintainability** | Adding user-friendly features like intuitive GUIs, “undo” options, and help systems makes the software easier to use but also more complex. It increases the amount of code and dependencies, which can make the software harder to maintain and test. |
| **Security** | **Performance** | Encryption, multi-factor authentication, and deep packet inspection add computational overhead and latency. |
| **Reliability** | **Portability** | Hardening software for a specific OS (using low-level kernel calls) makes it more stable but much harder to move to a different OS. |
| **Flexibility** | **Efficiency** | Using abstract layers or "plug-and-play" architectures allows for easy changes but often consumes more CPU and memory than |

# Proactive vs. Reactive Quality Management

In software development, quality can be managed in **two main ways**: proactively or reactively. **Proactive quality management**, also called **Quality Assurance (QA)**, focuses on **preventing defects before they occur**. This involves **defining coding standards, reviewing designs, performing risk analysis, and planning tests** before writing any code. By addressing potential problems early, QA reduces the likelihood of defects and minimizes the cost of fixing them later, because the **cost of quality rises exponentially the later a defect is discovered**.

**Reactive quality management**, also called **Quality Control (QC)**, focuses on **detecting and fixing defects after the software is built**. This includes **testing, debugging, and patching**. While QC is necessary, relying on it alone is less efficient and more costly, since defects discovered late can affect multiple parts of the system and may require significant rework.

# Proactive vs. Reactive Quality Management

Software quality cannot be considered in isolation;
it is constrained by the **Iron Triangle of Project Management**, which includes **scope, cost, and time**.

Increasing quality for example, by implementing higher security standards, better performance, or stricter usability requirements, will typically require **more budget, longer development time, or reducing the number of features delivered**.
Successful projects achieve a balance where quality goals are met **without exceeding budget or schedule**, by prioritizing features and planning carefully.

ing.a.haouari@gmail.com